

Lab 2 03/03/26

Program - 1

- 1) WAP in C to implement FCFS and SJF (preemptive & non-preemptive). Also compare FCFS and SJF.

FCFS:

```
#include <stdio.h>

int main() {
    int n;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], wt[n], tat[n], at[n], st[n], ct[n];

    for (int i = 0; i < n; i++) {
        printf("p%d Arrival Time: ", i+1);
        scanf("%d", &at[i]);

        printf("p%d Burst Time: ", i+1);
        scanf("%d", &bt[i]);
    }

    for (int i = 0; i < n-1; i++) {
        for (int j = i+1; j < n; j++) {
            if (at[i] > at[j]) {
                int temp;
                temp = at[i]; at[i] = at[j]; at[j] = temp;
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
            }
        }

        ct[i] = at[i] + bt[i];
        wt[i] = 0;
        st[i] = 0;
        tat[i] = bt[i];

        for (int i = 1; i < n; i++) {
```



P4 arrival time : 5

P4 burst time : 6

| Process | AT | BT | CT | TAT | LOT | RT |
|---------|----|----|----|-----|-----|----|
| P1      | 0  | 7  | 7  | 7   | 0   | 0  |
| P2      | 3  | 4  | 11 | 8   | 4   | 4  |
| P3      | 5  | 6  | 17 | 12  | 6   | 6  |
| P4      | 8  | 3  | 20 | 12  | 9   | 9  |

Average LOT = 4.75

Average TAT = 9.75

Average RT = 4.75

SJF :

#include <stdio.h>

#include <limits.h>

int main() {

int n, choice;

printf("Enter number of processes : ");

scanf("%d", &n);

int a[n], bt[n];

for (int i = 0; i < n; i++) {

printf("P%d Arrival time : ", i+1);

scanf("%d", &a[i]);

printf("P%d Burst Time : ", i+1);

scanf("%d", &bt[i]);

printf("In Choose Scheduling type : ");

printf("In 1. SJF Non-Preemptive");

printf("In 2. SJF Preemptive (SRTF)");

printf("In Enter choice : ");

scanf("%d", &choice);

switch(choice) {

```

case 1: {
    int completed[n], ct[n], wt[n], tat[n], st[n];
    int time = 0, count = 0;
    for (int i = 0; i < n; i++)
        completed[i] = 0;
    while (count < n) {
        int id = -1;
        int min = INT_MAX;
        for (int i = 0; i < n; i++) {
            if (ct[i] <= time && !completed[i] && bt[i] < min)
                min = bt[i];
            id = i;
        }
        if (id == -1) {
            time += bt[id];
            ct[id] = time;
            tat[id] = ct[id] - at[id];
            wt[id] = tat[id] - bt[id];
            st[id] = wt[id];
            completed[id] = 1;
            count++;
        } else {
            time++;
        }
    }
}

```

```

float avg_wt = 0, avg_tat = 0, avg_st = 0;
printf("\n Process \t AT \t BT \t CT \t TAT \t WT \t ST \n");
for (int i = 0; i < n; i++) {
    printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d \n", i+1, at[i],
        ct[i], tat[i], wt[i], st[i]);
    avg_wt += wt[i];
    avg_tat += tat[i];
    avg_st += st[i];
}

```

```

printf (" \n Average WT = %.2f", avg_wt/n);
printf (" \n Average TAT = %.2f", avg_tat/n);
printf (" \n Average RT = %.2f", avg_rt/n);
break;
}

```

case 2 :

```

int st[n], wt[n], tat[n], ct[n], response[n];

int first_exec[n];

for(int i=0; i<n; i++) {
    st[i] = bt[i];
    first_exec[i] = -1;
}

int complete=0, time=0;
while(complete < n) {
    int idn = -1;
    int min = INT_MAX;

    for(int i=0; i<n; i++) {
        if (st[i] <= time && st[i] > 0 && st[i] < min) {
            min = st[i];
            idn = i;
        }
        if (idn != -1) {
            if (first_exec[idn] == -1)
                first_exec[idn] = time;
            st[idn]--;
            time++;
        }
        if (st[idn] == 0) {
            complete++;
            ct[idn] = time;
            tat[idn] = ct[idn] - st[idn];
            wt[idn] = tat[idn] - bt[idn];
            response[idn] = first_exec[idn] - at[idn];
        }
    }
}

```

```

else {
    time++; }

```

```

float avg_wt = 0, avg_tat = 0, avg_rt = 0;
printf("In Process \t AT \t BT \t WT \t TAT \t RT \n");
for (int i = 0; i < n; i++)
    printf("P%d \t %d \t %d \t %d \t %d \t %d \n",
        i+1, at[i], bt[i], wt[i], tat[i], response[i]);

```

```

avg_wt += wt[i];
avg_tat += tat[i];
avg_rt += response[i];
}

```

```

printf("In Average WT = %.2f", avg_wt/n);
printf("In Average TAT = %.2f", avg_tat/n);
printf("In Average RT = %.2f", avg_rt/n);
break;
}

```

```

default:
    printf("Invalid choice \n");
}
return 0;
}

```

Output:

Enter number of processes: 4

P1 arrival: 0

P1 burst: 7

P2 arrival: 8

P2 burst: 3

P3 arrival: 3

P3 burst: 4

P4 arrival:

P4 burst:

Choose Sel

1. SJF Non

2. SJF P2

Enter cho

Process

P1

P2

P3

P4

Average

Average

Average

Compari

gth

3/3/

For

Aug

Aug

Aug

SRT

P4 arrival: 4.5

P4 burst: 6

Choose Scheduling Type:

1. SJF Non pre-emptive

2. SJF Pre-emptive

Enter choice: 1

| Process | AT | BT | WT | TAT | RT |
|---------|----|----|----|-----|----|
| P1      | 0  | 7  | 0  | 7   | 0  |
| P2      | 8  | 3  | 3  | 6   | 3  |
| P3      | 3  | 4  | 4  | 8   | 4  |
| P4      | 5  | 6  | 9  | 15  | 9  |

Average WT = 4.00

Average TAT = 9.00

Average RT = 4.00

FCFS -  $\left. \begin{array}{l} \text{WT} \\ \text{TAT} \\ \text{RT} \end{array} \right\}$  avg

Optimal

Comparison: FCFS takes more time

FCFS

SJF

SRTF

Avg WT = 4.75

4

4

Avg TAT = 9.75

9

9

Avg RT = 4.75

4

3.75

SRTF is more optimal

```

if(idn != -1) {
    if(p[idx].first == -1)
        p[idx].first = time;
    printf("P%d |", p[idx].pid);
    p[idx].st--;
    time++;
    if(p[idx].st == 0) {
        completed++;
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
    }
    else
        time++;
}

float avgwt = 0, avgtat = 0, avgst = 0;
printf("\n\nPID\t\tBT\t\tCT\t\tTAT\t\tWT\t\tRT\n");
for(i=0; i<n; i++) {
    int response = p[i].first - p[i].at;
    printf("P%d\t\t%d\t\t%d\t\t%d\t\t%d\t\t%d\t\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt,
        p[i].st, response);
    avgwt += p[i].wt;
    avgtat += p[i].tat;
    avgst += response;
}

printf("\n Average Waiting Time = %.2f", avgwt/n);
printf("\n Average Turnaround Time = %.2f", avgtat/n);
printf("\n Average Response Time = %.2f", avgst/n);
return 0;
}

```

Non pre-emptive:

$p[i].completed = 0;$

}

$printf("In Gantt Chart: \n");$

$while(completed < n)$

$highest = 9999;$

$idx = -1;$

$for(i=0; i < n; i++)$

$\{ if(p[i].at \leq time \ \&\& \ p[i].completed == 0)$

$\{ if(p[i].pr < highest)$

$\{ highest = p[i].pr;$

$idx = i;$

$\}}}$

$if(idx != -1)$

$\{ p[idx].st = time - p[idx].at;$

$printf("P %d |", p[idx].pid);$

$time += p[idx].bt;$

$p[idx].ct = time;$

$p[idx].tat = p[idx].ct - p[idx].at;$

$p[idx].wt = p[idx].tat - p[idx].bt;$

$p[idx].completed = 1;$

$completed++;$

}

else

$time++;$

}

$float avgwt = 0, avgtat = 0, avgst = 0;$

$printf("\n\nPID \t AT \t BT \t PR \t CT \t TAT \t WT$

$for(i=0; i < n; i++)$

$\{ printf("P %d \t %d \t %d \t %d \t %d \t %d \t %d \t %d \n",$

$p[i].pid, p[i].at, p[i].bt, p[i].pr, p[i].ct, p[i].tat, p[i].wt,$

$p[i].at);$

Output:

Enter number of processes: 4

Process 1

AT: 0

BT: 3

Priority: 2

Process 2

AT: 3

BT: 1

Priority: 4

Process 3

AT: 1

BT: 5

Priority: 6

Process 4

AT: 3

BT: 4

Priority: 3

Grant Chart:

|    |    |    |    |
|----|----|----|----|
| P1 | P4 | P2 | P3 |
| 0  | 3  | 7  | 8  |
|    |    |    | 13 |

| PID | AT | BT | PR | CT | TAT | WT | RT |
|-----|----|----|----|----|-----|----|----|
| P1  | 0  | 3  | 2  | 3  | 3   | 0  | 0  |
| P2  | 3  | 1  | 4  | 8  | 5   | 4  | 4  |
| P3  | 1  | 5  | 6  | 13 | 12  | 7  | 7  |
| P4  | 3  | 4  | 3  | 7  | 4   | 0  | 0  |

Average Waiting Time = 2.75

Average TAT = 6.00

Average Response Time = 2.75

Enter number of processes: 4

Process 1

AT: 1

BT: 3

P: 4

Process 2

AT: 4

BT: 3

P: 5

Process 3

AT: 2

BT: 1

P: 4

Process 4

AT: 6

BT: 3

P: 3

Gantt Chart:

| P4 | P4 | P4 | P1 | P1 | P3 | P2 | P2 | P2 |

| PID | AT | BT | Priority | CT | TAT | RT | WT |
|-----|----|----|----------|----|-----|----|----|
| P1  | 1  | 3  | 4        | 6  | 5   | 2  | 2  |
| P2  | 4  | 3  | 5        | 10 | 6   | 3  | 3  |
| P3  | 2  | 1  | 4        | 7  | 5   | 4  | 4  |
| P4  | 0  | 3  | 3        | 3  | 3   | 0  | 0  |

Average WT = 2.25

Average TAT = 4.75

Average RT = 2.25

Comparison :

Non preemptive

|    |    |    |    |
|----|----|----|----|
| P1 | P3 | P4 | P2 |
| 0  | 2  | 7  | 13 |

Preemptive

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| P1 | P2 | P3 | P4 | P4 | P4 | P4 | P4 | P4 | P3 | P3 | P3 | P3 | P2 | P2 | P2 | P1 |
| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |

Avg WT = 4.00

8.00

Avg TAT = 8.25

12.25

Avg RT = 4.00

0.00

~~Non~~ preemptive is more optimal

10/3

Lab-4

17/03/2026

Galin  
Silberschatz  
Gagne

9th ed

OS concepts

TB

Date  
Page

Round Robin

#include &lt;stdio.h&gt;

int main() {

int n, tq;

printf("Enter number of processes: ");

scanf("%d", &amp;n);

int at[n], bt[n], sem\_bt[n], ct[n], tat[n], wt[n], rt[n];

int first\_response[n];

for(int i=0; i&lt;n; i++) {

printf("Enter AT and BT for P%d: ", i+1);

scanf("%d %d", &amp;at[i], &amp;bt[i]);

sem\_bt[i] = bt[i];

first\_response[i] = -1; }

printf("Enter time quantum: ");

scanf("%d", &amp;tq);

int time = 0, completed = 0;

int gantt\_p[100], gantt\_t[100];

int idx = 0;

while(completed &lt; n) {

int done = 1;

for(int i=0; i&lt;n; i++) {

if(sem\_bt[i] &gt; 0 &amp;&amp; at[i] &lt;= time) {

done = 0;

if(first\_response[i] == -1) {

first\_response[i] = time;

rt[i] = time - at[i]; }

gantt\_p[idx] = i;

gantt\_t[idx] = time;

idx++;

```

if (sem - bt[i] > tq) {
    time += tq;
    sem - bt[i] -= tq;
} else {
    time += sem - bt[i];
    ct[i] = time;
    sem - bt[i] = 0;
    completed++;
}
}
}

```

```

if (done) {
    time++;
}
}

```

```

gantt t[idx] = time;
printf("\n Gantt chart : \n");
for (int i = 0; i < idx; i++) {
    printf(" P%d |", gantt - p[i] + 1);
}
printf("\n");

```

```

for (int i = 0; i < idx; i++)
    printf("-----");
printf("\n");
for (int i = 0; i <= idx; i++)
    printf("%-8d", gantt - t[i]);
printf("\n");

```

```

float total_wt = 0, total_tat = 0, total_at = 0;

```

```

printf("\n Process | t AT | t BT | t CT | t TAT | t WT | t RT \n");
for (int i = 0; i < n; i++) {
    tat[i] = at[i] - at[i];
    wt[i] = tat[i] - bt[i];
}

```



| Process | AT | BT | CT | TAT | WT | RT |
|---------|----|----|----|-----|----|----|
| P1      | 0  | 2  | 2  | 2   | 0  | 0  |
| P2      | 1  | 3  | 5  | 4   | 1  | 1  |
| P3      | 4  | 5  | 13 | 9   | 4  | 1  |
| P4      | 6  | 7  | 17 | 11  | 4  | 2  |

Average TAT = 6.50

Average WT = 2.25

Average RT = 1.00

② Enter number of processes: 5

Enter AT & BT for P1: 0

1

Average TAT = 10.20

Enter AT & BT for P2: 2

Average WT = 5.20

3

Average RT = 1.80

Enter AT & BT for ~~P3~~: 4

5

Enter AT & BT for P4: 6

7

Enter AT & BT for P5: 8

9

Enter time quantum: 4

If time quantum is small,  
response time is better,  
switching is more but  
TAT & WT increases.

~~8/3~~  
17/3/26

Gantt Chart:

| Grand Chess |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|
| P1          | P2 | P3 | P4 | P5 | P3 | P4 | P5 | P5 |    |
| 0           | 2  | 5  | 9  | 13 | 17 | 18 | 21 | 25 | 26 |

| Process | AT | BT | CT | TAT | WT | RT |
|---------|----|----|----|-----|----|----|
| P1      | 0  | 1  | 1  | 1   | 0  | 0  |
| P2      | 2  | 3  | 5  | 3   | 0  | 0  |
| P3      | 4  | 5  | 18 | 14  | 9  | 1  |
| P4      | 6  | 7  | 21 | 15  | 8  | 3  |
| P5      | 8  | 9  | 26 | 18  | 9  | 5  |

## Multi level queue

#include &lt;stdio.h&gt;

#include &lt;string.h&gt;

#define MAX 50

int main()

int n, i, time=0, completed=0, idn=0;

int pid[MAX], at[MAX], bt[MAX], st[MAX], <sup>int</sup>ct[MAX], tot[MAX],  
res[MAX], started[MAX]={0};

char type[MAX][10];

printf("Enter n: ");

scanf("%d", &amp;n);

for(i=0; i&lt;n; i++){

printf("In P %d\n", i+1);

scanf("%d %d %d %s", &amp;pid[i], &amp;at[i], &amp;bt[i], type[i]);

st[i] = bt[i];

}

for(i=0; i&lt;n-1; i++){

for(int j=i+1; j&lt;n; j++){

if(at[i] &gt; at[j]) {

int t;

t = at[i]; at[i] = at[j]; at[j] = t;

t = bt[i]; bt[i] = bt[j]; bt[j] = t;

t = pid[i]; pid[i] = pid[j]; pid[j] = t;

char tmp[100];

strcpy(tmp, type[j]);

strcpy(type[i], type[j]);

strcpy(type[j], tmp);

}

int sysQ[MAX], userQ[MAX], sf=0, sr=-1, uf=0, ur=-1;

int userend=-1;

int gantt[MAX], g=0;

```

while (completed < n) {
    while (idx < n && at[idx] <= time) {
        if (strcmp(type[idx], "system") == 0)
            sysA[++sf] = idx;
        else
            userA[++uf] = idx;
        idx++;
    }
}

```

```

if (current != -1 && strcmp(type[current], "user") == 0 && sf <= ss)
    userA[++uf] = current;
current = -1;
}

```

```

if (current == -1) {
    if (sf <= ss) current = sysA[sf++];
    else if (uf <= us) current = userA[uf++];
    else { time++; continue; }
}

```

```

if (!started[current]) {
    res[current] = time - at[current];
    started[current] = 1;
}
}

```

```

printf("In Time %d In System: ", time);
for (i = sf; i <= ss; i++) printf("P%d | ", pid[sysA[i]]);
if (sf > ss) printf("Empty");

```

```

printf("In User: ");
for (i = uf; i <= us; i++) printf("P%d | ", pid[userA[i]]);
if (uf > us) printf("Empty");
printf("\n");

```

```

    grant[i] = pid[current];
    rt[current]--;
    time++;
    if(rt[current] == 0) {
        ct[current] = time;
        tat[current] = ct[current] - at[current];
        wt[current] = tat[current] - bt[current];
        completed++;
        current = -1;
    }
    printf("\n Grant: \n");
    for(i=0; i<g; i++) printf("P%d |", grant[i]);
    printf("\n");
    for(i=0; i<=g; i++) printf("%d", i);

```

```

float aw=0, atot=0, ar=0;
printf("\n\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for(i=0; i<n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d", pid[i], at[i], bt[i], ct[i], tat[i], wt[i], res[i]);
    aw+= wt[i]; atot+= tat[i]; ar+= res[i];
}
printf("\n Avg WT= %.2f", aw/n);
printf("\n Avg TAT= %.2f", atot/n);
printf("\n Avg RT= %.2f", ar/n);
return 0;
}

```

Output:

Enter number of processes: 4

Process 1

PID: 1

AT: 0

BT: 2

Type: system

P2

PID: 2

AT: 3

BT: 4

Type: user

P3

PID: 3

AT: 4

BT: 4

Type: system

P4

PID: 4

AT: 5

BT: 5

Type: user

Time 0: P1 entered system queue

System: Empty

User: Empty

Time 1:

System: Empty

User: Empty

Time 13:

Time: 3  
P2 in user queue

S: E

S: Empty

U: E

U: Empty

Time 14:

Time: 4  
P3 entered system queue

S: E

S: Empty

U: E

U: P2

Time 15:

Time 5:  
P4 entered user queue

S: E

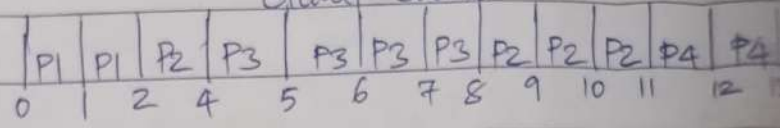
S: Empty

U: E

U: P2 | P4

Gantt Chart:

Time 6:



S: Empty

U: P2 | P4

Time 7:

S: Empty

U: P2 | P4

Time 8:

S: Empty

U: P4

Time 9:

S: Empty

U: P4

Time 10:

S: Empty

U: P4

Time 11:

S: E

U: E

Average WT = 2.50

Avg TAT = 6.25

Avg RT = 1.50

24/3

RTOS Scheduling

#include &lt;stdio.h&gt;

#define MAX 10

typedef struct {

int id, bt, deadline, period, share;

int ct, wt, tat, done;

} tprocess;

```
void reset(tprocess p[], int n) {
    for(int i=0; i<n; i++) {
        p[i].done=0;
    }
}
```

```
void EDF(tprocess p[], int n) {
    int time=0, completed=0;
    reset(p, n);
    while(completed < n) {
        int idx=-1;
        int mind=9999;
        for(int i=0; i<n; i++) {
            if(!p[i].done && p[i].deadline < mind) {
                mind = p[i].deadline;
                idx = i;
            }
        }
        time += p[idx].bt;
        p[idx].ct = time;
        p[idx].tat = p[idx].ct;
        p[idx].wt = p[idx].tat - p[idx].bt;
        p[idx].done = 1;
        completed++;
    }
}
```

printf("\n EDF scheduling \n");

printf("ID BT Deadline CT WT TAT\n");

for(int i=0; i&lt;n; i++) {

printf("P%d %d %d %d %d %d\n",

p[i].id, p[i].bt, p[i].deadline, p[i].ct, p[i].wt, p[i].tat);

```

}
void RMS(process p[], int n) {
    int time = 0, completed = 0;
    reset(p, n);
    while (completed < n) {
        int idn = -1;
        int minP = 9999;
        for (int i = 0; i < n; i++) {
            if (!p[i].done && p[i].period < minP) {
                minP = p[i].period;
                idn = i;
            }
        }
        time += p[idn].bt;
        p[idn].ct = time;
        p[idn].tat = p[idn].ct;
        p[idn].wt = p[idn].tat - p[idn].bt;
        p[idn].done = 1;
        completed++;
    }
    printf("\n Rate Monotonic Scheduling\n");
    printf("ID  BT  Period  CT  WT  TAT\n");
    for (int i = 0; i < n; i++) {
        printf("%d %d %d %d %d %d\n", p[i].id, p[i].period, p[i].ct, p[i].wt, p[i].tat);
    }
}

```

```

int checkRM(process p[], int n) {
    float U = 0;
    for (int i = 0; i < n; i++)
        U += (float) p[i].bt / p[i].period;
    float bound = n * pow(2, (float) 1/n) - 1;
    printf("\n RM Utilization = %.3f, Bound = %.3f\n", U, bound);
    if (U <= bound)

```

```
return 1;
```

```
else
```

```
return 0; }
```

```
int checkEDF(Process p[], int n) {
```

```
float U=0;
```

```
for(int i=0; i<n; i++)
```

```
U += (float) p[i].bt / p[i].period;
```

```
printf("EDF Utilization = %.3f\n", U);
```

```
if (U <= 1.0)
```

```
return 1;
```

```
else
```

```
return 0;
```

```
}
```

```
int checkPRP(Process p[], int n) {
```

```
int total=0;
```

```
for(int i=0; i<n; i++)
```

```
total += p[i].share;
```

```
printf("Proportional Total share = %.d\n", total);
```

```
if (total > 0)
```

```
return 1;
```

```
else
```

```
return 0;
```

```
}
```

```
void Proportional(Process p[], int n) {
```

```
int time=0, completed=0;
```

```
sort(p, n);
```

```
while(completed < n) {
```

```
int idn=-1;
```

```
int maxshare=-1;
```

```

for(int i=0; i<n; i++) {
    if(1) { P[i] done & P[i].share & markshare; }
    markshare = P[i].share;
    id = i; }
    time += P[id].bt;
    P[id].ct = time;
    P[id].bt = P[id].bt;
    P[id].act = P[id].bt - P[id].bt;
    P[id].done = 1;
    completed++;
    printf("%d Processual %d\n", id);
    printf("%d BT share at act %d\n", id);
    if(id == 0; i < n; i++) {
        printf("%d %d %d %d %d %d\n", id, P[id].bt, P[id].share, P[id].ct, P[id].act, P[id].done);
    }
}

int main() {
    int n;
    Process P[100];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter process details\n");
    for(int i=0; i<n; i++) {
        P[i].id = i;
        printf("Process %d: %d, %d, %d, %d, %d, %d\n", i, P[i].id, P[i].bt, P[i].share, P[i].ct, P[i].act, P[i].done);
        printf("Enter Time: ");
        scanf("%d", &P[i].bt);
        printf("Deadline: ");
        scanf("%d", &P[i].deadline);
        printf("Period: ");
        scanf("%d", &P[i].period);
    }
}

```

```

print("Size:");
sort("x", &PII, size);
if (checkRM (P, n)) {
    print("RM: Schedulable in");
    RM(P, n);
}
else
    print("RM: Not schedulable in");

if (checkEDF (P, n)) {
    print("EDF: Schedulable in");
    EDF(P, n);
}
else
    print("EDF: Not schedulable in");

if (checkFP (P, n)) {
    print("FP: Not Valid in");
    FP(P, n);
}
else
    print("FP: Invalid in");

subm 0;
}

PMS }
EDF }

```

Output:

Total number of processes: 2

Enter process details:

Process 0:

ET: 3

Deadline: 5

Period: 10

Step 1:

Process: 1

BT: 1

Deadline: 4

Period: 10

Score: 2

RM Utilization = 0.400      Found = 0.828

RM: Schedulable

Rate Monotonic Scheduling

| ID             | BT | Period | CT | WT | TAT |
|----------------|----|--------|----|----|-----|
| P <sub>0</sub> | 3  | 10     | 3  | 0  | 3   |
| P <sub>1</sub> | 1  | 10     | 4  | 3  | 4   |

EDF Utilization = 0.400

EDF: Schedulable

EDF

| ID             | BT | Deadline | CT | WT | TAT |
|----------------|----|----------|----|----|-----|
| P <sub>0</sub> | 3  | 5        | 4  | 1  | 4   |
| P <sub>1</sub> | 1  | 4        | 1  | 0  | 1   |

Proportional total score = 3

Proportional: Valid

Proportional

| ID             | BT | Score | CT | WT | TAT |
|----------------|----|-------|----|----|-----|
| P <sub>0</sub> | 3  | 1     | 4  | 1  | 4   |
| P <sub>1</sub> | 1  | 1     | 0  | 1  | 1   |

Enter number of processes: 3

P0:

BT: 3

Deadline: 4

Period: 5

Slack: 3

P1

BT: 5

Deadline: 4

Period: 4

Slack: 3

P2

BT: 6

Deadline: 6

Period: 6

Slack: 6

RM  $V = 2.850$  Round = 0.780

RM: Not schedulable

EDF  $V = 2.850$

EDF: Not schedulable

Preemptive Total Slack = 12

Scheduling = Valid

Preemptive

| ID | BT | Slack | CT | WT | TAT |
|----|----|-------|----|----|-----|
| P0 | 3  | 3     | 9  | 6  | 9   |
| P1 | 5  | 3     | 14 | 9  | 14  |
| P2 | 6  | 6     | 6  | 0  | 6   |

3/13

Q) The following program consists of 3 concurrent processes and 2 binary semaphores. The semaphores are initialized as  $S_0=1$ ,  $S_1=0$ ,  $S_2=0$ . Find out how many times process P0 will print '0'. Assume the order of execution as P0, P1, P2, P0, P1.

P0:

P1:

P2:

```
while(true){
    wait(S0);
    wait(S1);
    wait(S2);
    printf("0");
    signal(S1);
    signal(S2);
}
```

Initial  $S_0=1$  $S_1=0$  $S_2=0$ 

Process

P0

0

1

1

0

P1

1

0

1

—

P2

1

0

0

—

P0

0

1

1

00

P1

1

0

1

—

P0 will print 0 2 times

Q) Consider 2 processes A & B. Two semaphore variables S & T are used to synchronize the processes. S is initialized to 0 and T is initialized to 1. Processes are scheduled in the order ABAABAAAB. What will be printed?

Page

B

```
while(1) {
    read(T)
    print T
    print I
    signal(S)
}
```

valide (D)  
point (S)  
point P  
point P  
signal (T)

Pennak (1875)

~~Process A is~~

III

英寸 PP

IIPII

Process B is blocked

11 PP PP

Bo A is Blocked

11 PP 11 PP 11

pppppp

Final

2

$$\text{SeO}$$

A

at

△

7

3

4

4

11

4

Revised Buffer: Producer Consumer Problem

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <semaphore.h>
```

```
#include <semaphore.h>
```

```
#define size size 5
```

```
int buffer[size];
```

```
int in=0, out=0;
```

```
sem_t empty, full, mutex;
```

```
void * producer(void * arg) {
```

```
    int item;
```

```
    for(int i=0; i<15; i++) {
```

```
        item=i;
```

```
        sem_wait(&empty);
```

```
        sem_wait(&mutex);
```

```
        buffer[in] = item; printf("Produced: %d at buffer[%d]\n", item,
```

```
in=(in+1)%size;
```

```
        sem_post(&mutex);
```

```
        sem_post(&full);
```

```
    }
```

```
pthread_exit(NULL); }
```

```
void * consumer(void * arg) {
```

```
    int item;
```

```
    for(int i=0; i<15; i++) {
```

```
        sem_wait(&full);
```

```
        sem_wait(&mutex);
```

```
        item = buffer[out];
```

```
        printf("Consumed: %d from buffer[%d]\n", item, out);
```

```
        out = (out+1)%size;
```

```
        sem_post(&mutex);
```

```
sem - post (& empty);
```

```
}
```

```
pthread - exit (NULL); }
```

```
int main() {
```

```
pthread - t p, c;
```

```
sem - init (&empty, 0, size);
```

```
sem - init (&full, 0, 0);
```

```
sem - init (&mutex, 0, 1);
```

```
pthread - create (&p, NULL, producer, NULL);
```

```
pthread - create (&c, NULL, consumer, NULL);
```

```
pthread - join (p, NULL);
```

```
pthread - join (c, NULL);
```

```
sem - destroy (&empty);
```

```
sem - destroy (&full);
```

```
sem - destroy (&mutex);
```

```
return 0;
```

```
}
```

Output:

p c p c

Produced 0 at buffer[0]

Bgs [-1 -1 -1 -1]

Produced 1 at buffer[1]

Consumed 0 from buffer[0]

Produced 2 at buffer[2]

Consumed 1 from buffer[1]

Produced 3 at buffer[3]

Consumed 2 from buffer[2]

Produced 4 at buffer[4]

Consumed 3 from buffer[3]

Produced 5 from buffer[0]

Consumed 4 from buffer[4]

PCPCPCP

Index: PCPCPC

buffer size 5, in=0, out=0

empty=5

full=0

under=1

0: empty=4 under=0

1: empty=3 under=0

buffer[0]=0 in=1

buffer[1]=1 in=2

under=1 full=1

under=1 full=2

buffer=[0 - - -]

buffer=[0 1 - -]

0: full=1 under=0

2: empty=3 under=0

out=1

buffer[2]=2 in=3

under=1 empty=4

under=1 full=2

buffer=[1 - -]

buffer=[1 2 -]

1: full=1 under=0

3: empty=3 under=0

out=2

buffer[3]=3 in=4

under=1 empty=4

under=1 full=2

buffer=[-1 - -]

buffer=[- 2 3 -]

## Dining Philosophers Problem

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
#define N 5
```

```
sem_t forks[N],
```

```
sem_t room;
```

```
void *philosopher(void *num) {
```

```
    int id = *(int *)num;
```

```
    while(1) {
```

```
        printf("Philosopher %d is thinking\n", id);
```

```
        sleep(1);
```

```
        sem_wait(&room);
```

```
        sem_wait(&forks[id]);
```

```
        printf("Philosopher %d picked left fork %d\n", id, id);
```

```
        sem_wait(&forks[(id+1)%N]);
```

```
        printf("Philosopher %d picked right fork %d\n", id, (id+1)%N);
```

```
        printf("Philosopher %d is eating\n", id);
```

```
        sleep(2);
```

```
        sem_post(&forks[id]);
```

```
        sem_post(&forks[(id+1)%N]);
```

```
        printf("Philosopher %d put down forks %d & %d\n",
```

```
              (id+1)%N,
```

```
              sem_post(&room);
```

```
        sleep(1); }
```

```
int main() {
```

```
    pthread_t threads[N];
```

```
    int ids[N];
```

```
    sem_init(&room, 0, N-1);
```

```
    for(int i=0; i<N; i++) {
```

```
        sem_init(&forks[i], 0, 1); }
```

```
for(int i=0; i<N; i++) {
    id[i]=i;
    pthread_create(&thead[i], NULL, philosopher, &id[i]);
}
```

```
for(int i=0; i<N; i++) {
    pthread_join(thead[i], NULL);
}
```

return 0; }

Output:

```
philosopher 1 is thinking
P 3 is thinking
P 4 is thinking
P 2 is thinking
P 0 is thinking
P 0 picked left fork 0
P 3 picked left fork 3
P 0 picked right fork 1
P 0 is eating
P 4 picked left fork 4
P 0 put down forks 0 and 1
```

Burke's Algorithm : Safety Algorithm & Resource request

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10
```

```
int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int allocation[MAX][MAX], max[MAX][MAX], need[MAX][MAX];
    int available[MAX];
```

```
printf("\nEnter allocation matrix\n");
for(int i=0; i<n; i++) {
    for(int j=0; j<m; j++) {
        scanf("%d", &allocation[i][j]);
    }
}
```

```
printf("\nEnter max matrix\n");
for(int i=0; i<n; i++) {
    for(int j=0; j<m; j++) {
        scanf("%d", &max[i][j]);
    }
}
```

```
printf("\nEnter available resources\n");
for(int i=0; i<m; i++) {
    scanf("%d", &available[i]);
}
```

```
for(int i=0; i<n; i++) {
    for(int j=0; j<m; j++) {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}
```

```
if process; int request[Max];
int need[Max];
```

```
bool finish[Max];
```

```
int safeSequence[Max];
```

```
int count = 0;
```

→ Request  
request

```
for (int i = 0; i < m; i++)
    need[i] = available[i];
```

// Process request

```
printf("\nEnter process number  
making request: ");
```

```
scanf("%d", &process);
```

```
printf("\nEnter request vector: ");
```

```
for (int i = 0; i < m; i++)
```

```
    scanf("%d", &request[i]);
```

```
for (int i = 0; i < m; i++) {
```

```
    if (request[i] > need[process][i]) {
```

```
        printf("\nError: Request exceeds  
maximum need\n");
```

```
    return 0; }
```

```
while (count < n) {
    bool found = false;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (finish[i] == false) {
```

```
            int j;
```

```
            for (j = 0; j < m; j++) {
```

```
                if (need[i][j] > need[j][j])
```

```
                    break;
```

```
            return 0; }
```

```
        for (int i = 0; i < m; i++) {
```

```
            available[i] -= request[i];
```

```
            allocation[process][i] += request[i];
```

```
            need[process][i] -= request[i]; }
```

```
        // Rollback
```

```
        for (int i = 0; i < m; i++) {
```

```
            available[i] += request[i];
```

```
            allocation[process][i] -= request[i];
```

```
            need[process][i] += request[i]; }
```

```
        printf("\nRequest cannot be granted\n");
```

```
    if (found == false) {
        printf("\nSystem will be in UNSAFE state\n");
```

```
        return 0; }
```

```
    printf("\n Request can be granted\n");
```

```
    printf("\n System is in Safe state\n Safe sequence: ");
```

```
for(int i=0; i<n; i++) {
    printf("P%d", sequence[i]);
    if(i != n-1)
        printf(" -> ");
    printf("\n");
}
return 0; }
```

Output:

Enter number of processes: 5  
Enter number of sequences: 3

Enter allocation matrix:

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 0 | 4 | 3 |
| 2 | 0 | 0 | 1 | 2 |
| 3 | 0 | 2 |   |   |
| 2 | 1 | 1 |   |   |
| 0 | 0 | 2 |   |   |

Enter max matrix:

|   |   |   |
|---|---|---|
| 7 | 5 | 3 |
| 3 | 2 | 2 |
| 9 | 0 | 2 |
| 2 | 2 | 2 |
| 4 | 3 | 3 |

Enter available resources:

3 3 2

System is in safe state

Safe sequence: P1 -> P3 -> P4 -> P0 -> P2

Enter allocation matrix:

|   |   |   |
|---|---|---|
| 0 | 1 | 0 |
| 2 | 0 | 0 |
| 3 | 0 | 2 |
| 2 | 1 | 1 |
| 0 | 0 | 2 |

Enter max matrix:

|   |   |   |
|---|---|---|
| 1 | 5 | 3 |
| 3 | 2 | 2 |
| 9 | 0 | 2 |
| 2 | 2 | 2 |
| 4 | 3 | 3 |

Enter available resources:

|   |   |   |
|---|---|---|
| 3 | 3 | 2 |
|---|---|---|

Enter process number making request: 2

Enter request vector:

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
|---|---|---|

~~Enter: Request exceeds maximum need.~~

5/5/24

Lab-8

classmate

Date  
Page

## Deadlock Detection

```
#include <stdio.h>
```

```
int main() {
```

```
    int n,m,i,j;
```

```
    printf("\nEnter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("\nEnter no of resource types: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], request[n][m], available[m];
```

```
    int finish[n];
```

```
    printf("\nEnter allocation matrix: \n");
```

```
    for(i=0; i<n; i++) {
```

```
        for(j=0; j<m; j++) {
```

```
            scanf("%d", &alloc[i][j]); }
```

```
    printf("\nEnter request matrix: \n");
```

```
    for(i=0; i<n; i++) {
```

```
        for(j=0; j<m; j++) {
```

```
            scanf("%d", &request[i][j]); }
```

```
    printf("\nEnter available resources: \n");
```

```
    for(i=0; i<m; i++) {
```

```
        scanf("%d", &available[i]); }
```

```
    for(i=0; i<m; i++) {
```

```
        check[i] = available[i]; }
```

```
    for(i=0; i<n; i++) {
```

```
for (j=0; j<m; j++) {
    if (alloc[i][j] != 0) {
        zero = 0;
        break;
    }
}
```

```
if (zero)
    finish[i] = 1;
else
    finish[i] = 0;
}
```

```
int found;
do {
    found = 0;
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            int can_alloc = 1;

```

```
for (j=0; j<m; j++) {
    if (request[i][j] > alloc[j]) {
        can_alloc = 0;
        break;
    }
}
```

```
if (can_alloc) {
    for (j=0; j<m; j++) {
        work[j] += allocation[i][j];
    }
    finish[i] = 1;
    found = 1;
}
}
```

```
int deadlock = 0;
printf("\n Deadlocked processes: %d", n);
for (i=0; i<n; i++) {
    if (finish[i] == 0) {

```

```

printf("%f.%d", i);
deadlock=1; }
if (deadlock==0)
printf("No deadlock detected"); }
printf("\n");
return 0; }

```

Output:

Enter no of processes: 4

Enter no of resource types: 3

Enter allocation matrix:

0 1 0

2 0 0

3 0 2

2 1 1

Enter request matrix:

0 0 2

2 3 1

1 1 0

0 2 1

Enter available resources:

3 3 2

Ans

Deadlocked processes:

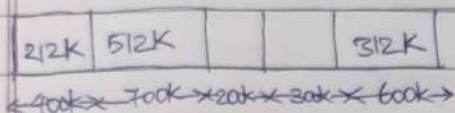
No deadlock detected

Lab - 9  
12/05/26

## Memory allocation techniques

- Q) Given ~~you~~ have <sup>a free</sup> ~~5~~ <sup>in order</sup> memory partitions 400k, 700k, 200k, 300k & 600k, how would each of first fit, best fit & worst fit algorithms work to place processes of 212k, 512k, 312k, 526k in order

First fit



P<sub>1</sub> - 212K

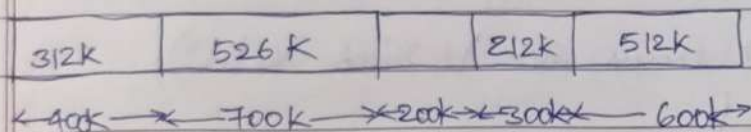
P<sub>2</sub> - 512K

P<sub>3</sub> - 312K

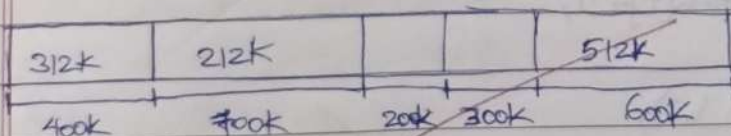
P<sub>4</sub> - 526K

526K cannot be allocated memory

Best Fit



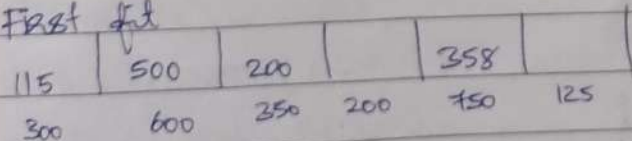
Worst Fit



526K cannot be allocated memory

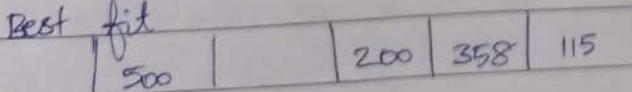
- Q) Memory partition: 300KB, 600KB, 350KB, 200KB, 750KB, 125KB  
Process size: 115KB, 500KB, 358KB, 200KB, ~~375KB~~

First fit



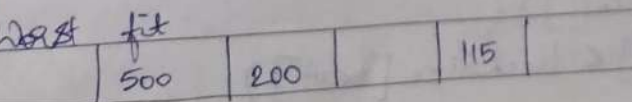
375 Not allocated

Best fit



375 NA

Worst fit



358, 375 NA

Code:

```
#include <stdio.h>
```

```
void first(int blockSize[], int m, int processSize[], int n) {
    int allocation[n];
```

```
    for(int i=0; i<n; i++)
```

```
        allocation[i] = -1;
```

```
    for(int i=0; i<n; i++) {
```

```
        for(int j=0; j<m; j++) {
```

```
            if (blockSize[j] >= processSize[i]) {
```

```
                allocation[i] = j;
```

```
                blockSize[j] = -1;
```

```
                break; } }
```

```
    printf("First fit\n");
```

```
    printf("Process no\t Process size\t Block No\n");
```

```
    for(int i=0; i<n; i++) {
```

```
        printf("%d\t\t\t\t\t", i+1, processSize[i]);
```

```
        if (allocation[i] != -1)
```

```
            printf("%d\n", allocation[i]+1);
```

```
        else
```

```
            printf("Not allocated\n");
```

```
    }
```

```
void Best(int blockSize[], int m, int processSize[], int n)
```

```
{
```

```
    int allocation[n];
```

```
    for(int i=0; i<n; i++) {
```

```
        allocation[i] = -1;
```

```
    for(int i=0; i<n; i++) {
```

```
        int best = -1;
```

```
        for(int j=0; j<m; j++) {
```

```
            if (blockSize[j] >= processSize[i]) {
```

```
                if (best == -1 || blockSize[j] < blockSize[best])
```

```

best = j; }
if (best != -1) {
    allocation[i] = best;
    blockSize[best] = -1; }
printf("Best Fit\n");
printf("Process No \t Process Size \t Block No\n");
for (int i = 0; i < n; i++) {
    printf("%d \t \t %d \t \t", i+1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i]+1);
    else
        printf("Not allocated\n"); }
}

```

```

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int worst = -1;
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (worst == -1 || blockSize[j] > blockSize[worst])
                    worst = j; }
            }
        if (worst != -1) {
            allocation[i] = worst;
            blockSize[worst] = -1; }
        printf("Worst Fit\n");
        printf("Process No \t Process Size \t Block No\n");
        for (int i = 0; i < n; i++) {
            printf("%d \t \t %d \t \t", i+1, processSize[i]);

```

```

if (allocation[i] != -1)
    printf("%d\n", allocation[i] + 1);
else
    printf("Not allocated\n");
}

```

```

int main() {
    int m, n;
    printf("Enter no of memory blocks: ");
    scanf("%d", &m);
    int block[m], block b1[m], b2[m];
    printf("Enter block sizes: \n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &block[i]);
        b1[i] = b2[i] = block[i];
    }

```

```

    printf("Enter no of processes: ");
    scanf("%d", &n);
    int process[n];
    printf("Enter process sizes: \n");
    for (int i = 0; i < n; i++)
        scanf("%d", &process[i]);
    for First(block, m, process, n);
    Best(b1, m, process, n);
    Worst(b2, m, process, n);
    return 0;
}

```

Output:

```

Enter no of memory blocks: 6
Enter block sizes:
300
600

```

350

200

750

125

Enter no of processes: 5

Enter process sizes:

115

500

358

200

375

First fit:

| PNo | P size | Block No      |
|-----|--------|---------------|
| 1   | 115    | 1             |
| 2   | 500    | 2             |
| 3   | 358    | 5             |
| 4   | 200    | 3             |
| 5   | 375    | Not Allocated |

Best fit:

| PNo | PSize | Block No      |
|-----|-------|---------------|
| 1   | 115   | 6             |
| 2   | 500   | 2             |
| 3   | 358   | 5             |
| 4   | 200   | 4             |
| 5   | 375   | Not allocated |

Worst fit:

| PNo | PSize | Block No      |
|-----|-------|---------------|
| 1   | 115   | 5             |
| 2   | 500   | 2             |
| 3   | 358   | Not allocated |
| 4   | 200   | 3             |
| 5   | 375   | Not allocated |

26/5/26

classmate

Date

Page

Page replacement algorithm :

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int pages[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5}
```

```
    int n = 12, f = 3;
```

```
    int frames[3], i, j, k, hit, faults;
```

```
    for(i = 0; i < f; i++) frames[i] = -1;
```

```
    faults = 0; k = 0;
```

```
    printf("FIFO\n");
```

```
    for(i = 0; i < n; i++) {
```

```
        hit = 0;
```

```
        for(j = 0; j < f; j++)
```

```
            if(frames[j] == pages[i])
```

```
                hit = 1;
```

```
        if(!hit) {
```

```
            frames[k] = pages[i];
```

```
            k = (k + 1) % f;
```

```
            faults++; }
```

```
        printf("%d → ", pages[i]);
```

```
        for(j = 0; j < f; j++)
```

```
            frames[j] == -1 ? printf("- ") : printf("%d ", frames[j]);
```

```
        printf(hit ? "Hit\n" : "Miss\n"); }
```

```
    printf("Page Faults = %d\n", faults);
```

```
    int time[3] = {0}, count = 0, pos;
```

```
    for(i = 0; i < f; i++) frames[i] = -1;
```

```
    faults = 0;
```

```
    printf("LRU\n");
```

```
    for(i = 0; i < n; i++) {
```

```
        hit = 0;
```

```
        for(j = 0; j < f; j++) {
```

```

if (frames[j] == pages[i]) {
    if (!hit) hit = 1;
    time[j] = ++count; }
if (!hit) {
    pos = 0;
    for (j = 1; j < f; j++)
        if (time[j] < time[pos])
            pos = j;
    frames[pos] = pages[i];
    time[pos] = ++count;
    faults++; }
printf("%d → ", pages[i]);
for (j = 0; j < f; j++)
    frames[j] == -1 ? printf("-") : printf("%d ", frames[j]);
printf(hit ? "Hit\n" : "Miss\n"); }
printf("Page Faults = %d\n\n", faults);

```

```

for (i = 0; i < f; i++) frames[i] = -1;
faults = 0;
printf("Optimal\n");
for (i = 0; i < n; i++) {
    hit = 0;
    for (j = 0; j < f; j++)
        if (frames[j] == pages[i])
            hit = 1;
    if (!hit) {
        int farthest = -1, next;
        pos = 0;
        for (j = 0; j < f; j++) {
            next = 999;
            for (k = i + 1; k < n; k++)
                if (frames[j] == pages[k])

```

```

{ next = k;
  break; }
if (next > farthest) {
  farthest = next;
  pos = j; }
frames[pos] = pages[i];
faults++;
printf("%d → ", pages[i]);
for (j = 0; j < f; j++)
  frames[j] == -1 ? printf("- ") : printf("%d", frames[j]);
printf("Hit ? "Hit\n" : "Miss\n");
printf("Page Faults = %d\n", faults);
return 0; }

```

Output:

FIFO:

1 → 1 — Miss

2 → 1 2 — Miss

3 → 1 2 3 Miss

4 → 4 2 3 Miss

1 → 4 1 3 Miss

2 → 4 1 2 Miss

5 → 5 1 2 Miss

1 → 5 1 2 Hit

2 → 5 1 2 Hit

3 → 5 3 2 Miss

4 → 5 3 4 Miss

5 → 5 3 4 Hit

Page Faults = 9

LRU

1 → 1 - - Miss

2 → 1 2 - Miss

3 → 1 2 3 Miss

4 → 4 2 3 Miss

1 → 4 1 3 Miss

2 → 4 1 2 Miss

5 → 5 1 2 Miss

1 → 5 1 2 Hit

2 → 5 1 2 Hit

3 → 3 1 2 Miss

4 → 3 4 2 Miss

5 → 3 4 5 Miss

Page faults = 10

Optimal

1 → 1 - - Miss

2 → 1 2 - Miss

3 → 1 2 3 Miss

4 → 1 2 4 Miss

1 → 1 2 4 Hit

2 → 1 2 4 Hit

5 → 1 2 5 Miss

1 → 1 2 5 Hit

2 → 1 2 5 Hit

3 → 3 2 5 Miss

4 → 4 2 5 Miss

5 → 4 2 5 Hit

Page faults = 7

es[j]);

8/2

26/1